An Exact Penalty Matrix for Cubic Smoothing Splines on Arbitrary Knot Vectors

Aadya Chinubhai

August 14, 2026

Abstract

These notes derive the penalty matrix Ω of a cubic smoothing spline from first principles, for an arbitrary knot vector. Starting from the definition of the B-spline basis, we express differentiation as a matrix, integrate the squared second derivative in closed form, and arrive at an exact factorization $\Omega = C^\top R C$ built from the knot vector alone. Later sections derive the generalized cross-validation criterion for selecting the smoothing parameter, analyze the spectrum of the influence matrix, examine conditioning, and survey related software.

Contents

1 The Basics

A cubic spline on a knot vector t is a linear combination of cubic B-spline basis functions,

$$f(u) \;=\; \sum_{j=1}^{m} c_j \, B_j(u), \tag{1}$$

where B_j are the basis functions, $\boldsymbol{c} = (c_1, \ldots, c_m)^\top$ are the coefficients, and m is the number of basis functions supported by t.

The roughness penalty is the integrated squared curvature of f,

$$\int \big(f''(u)\big)^2 \, \mathrm{d}u, \tag{2}$$

which, substituting (1), is a quadratic form in the coefficients: $\boldsymbol{c}^\top \Omega \, \boldsymbol{c}$ with

$$\Omega_{ij} \;=\; \int B_i''(u) \, B_j''(u) \, \mathrm{d}u. \tag{3}$$

Computing Ω exactly, for an arbitrary knot vector t, is the subject of this document.

The boundary knots are repeated four times (the order of the spline). The repeats leave zero width between them, so the first basis function has no interval over which to increase from zero the way an interior one does, and, since each repetition also removes one continuity requirement at that point, nothing forces it to. The basis function therefore attains its maximum at the boundary knot itself: $B_1(t_{\min}) = 1$, with every other basis function zero there. The spline is therefore pinned to its first coefficient, $f(t_{\min}) = c_1$, and likewise $f(t_{\max}) = c_m$ at the right end.

Symbols used in this document

Before deriving anything, here is every symbol that appears, in order of first use. The running example throughout is the knot vector $t = (0, 0, 0, 0,\ 0.7,\ 2.3,\ 3, 3, 3, 3)$.

symbol	what it is	size (example)
t	knot vector, boundary knots repeated 4×	10 knots
K	number of interior knots	2
$m = K + 4$	number of cubic basis functions	6
u	the free variable the spline is a function of	–
$B_j(u)$	j-th cubic B-spline basis function	m of them
$f(u)$	the spline, $f = \sum_j c_j B_j$	–
$\boldsymbol{c}$	coefficients of f in the cubic basis	$m = 6$
Ω	penalty matrix, $\Omega_{ij} = \int B_i'' B_j''$	$m \times m$
D_1	differentiation matrix, cubic → quadratic coeffs	7×6
D_2	differentiation matrix, quadratic → linear coeffs	8×7
$C = D_2 D_1$	second-derivative matrix, $\boldsymbol{c} \to$ coeffs of f''	8×6
$N_p(u)$	p-th hat function (linear B-spline)	$m + 2 = 8$ of them
$\boldsymbol{\gamma} = C\boldsymbol{c}$	coefficients of f'' in the hat basis	$m + 2 = 8$
R	hat mass matrix, $R_{pq} = \int N_p N_q$	8×8, tridiagonal
h, h'	widths of knot intervals	–
Used only in the solve and validation:		
$(x_i, y_i), w_i$	the n data points and their weights	n each
X	design matrix, $X_{ij} = B_j(x_i)$	$n \times m$
W	diagonal matrix of the weights	$n \times n$
λ	smoothing parameter	scalar
$\boldsymbol{g}$	Greville points, $g_j = (t_{j+1} + t_{j+2} + t_{j+3})/3$	m

The logic of the document in one line: Section 2 builds C, Section 3 builds R, and Section 4 combines them into $\Omega = C^\top R\, C$.

2 Differentiation as a matrix

Start from what a cubic spline is on one interval: an ordinary cubic polynomial $au^3 + bu^2 + cu + d$. Differentiate it once and you get a quadratic; differentiate again and you get a straight line. This stays true piece by piece across the whole spline: *the derivative of a cubic spline is a quadratic spline, and its second derivative is a piecewise-linear function*, and both live on the same knots, since differentiation does not move the break points.

Now, our spline is not written as a, b, c, d per piece; it is written as coefficients $c_1, \ldots, c_6$ in the B-spline basis (eq. (1)). So the question becomes: if I know the six numbers describing f, what are the seven numbers describing f' in the quadratic B-spline basis? (Seven, because our ten knots carry six cubic but seven quadratic basis functions; see the counting table of Section 2.)

De Boor's formula (de Boor, 2001, p. 117) answers this: each new coefficient is a *slope* computed from two neighbouring old ones,

$$\text{new coefficient } j \;=\; 3 \cdot \frac{c_j - c_{j-1}}{t_{j+3} - t_j}. \tag{4}$$

Interpretation: $c_j - c_{j-1}$ is a rise between neighbouring control heights, $t_{j+3} - t_j$ is the horizontal distance those two coefficients are anchored over, and the factor 3 is the same 3 as in $(u^3)' = 3u^2$.

Since every new coefficient is a weighted difference of two old ones, the whole operation is a matrix. Row j of that matrix selects c_{j-1} and c_j with weights $\mp d_j$, where $d_j = 3/(t_{j+3} - t_j)$. For

our ten knots $t = (0, 0, 0, 0,\ 0.7,\ 2.3,\ 3, 3, 3, 3)$ the matrix is 7×6 (in general $(m+1) \times m$), six coefficients in, seven out:

$$D_1 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ -4.29 & 4.29 & 0 & 0 & 0 & 0 \\ 0 & -1.30 & 1.30 & 0 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1.30 & 1.30 & 0 \\ 0 & 0 & 0 & 0 & -4.29 & 4.29 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

For instance the 4.29 in the second row is $3/(t_5 - t_2) = 3/0.7$. The first and last rows are zero because their run $t_{j+3} - t_j$ is the distance across the repeated boundary knots, which is zero. There is no width for a slope to live on; this is the clamping of Section 2 seen from the coefficient side.

Doing the same again (quadratics → lines, with a 2 in place of the 3 and a two-knot run in the denominator) gives an 8×7 (in general $(m+2) \times (m+1)$) matrix D_2, and chaining the two

$$C = D_2 D_1 \ \in \ \mathbb{R}^{(m+2)\times m} \qquad (8 \times 6 \text{ here}), \tag{5}$$

takes the six numbers describing f straight to the eight numbers describing f'' as a piecewise-linear function, in the basis of *hat functions* on the same knots:

$$C = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 12.24 & -15.97 & 3.73 & 0 & 0 & 0 \\ 0 & 1.13 & -2.00 & 0.87 & 0 & 0 \\ 0 & 0 & 0.87 & -2.00 & 1.13 & 0 \\ 0 & 0 & 0 & 3.73 & -15.97 & 12.24 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Each row of C combines *three* neighbouring coefficients with alternating signs: a second difference, which is exactly what one would expect from a matrix whose job is to take a second derivative.

Two things to notice, because the whole construction rests on them. First, C was built from the knot vector alone: no data point x_i or y_i appears anywhere in this section. Second, everything was exact: eq. (4) is an identity, not an approximation.

3 Integrating the squared second derivative: hat functions

What kind of function is f''?

Go back to a single piece of the spline, the cubic $au^3 + bu^2 + cu + d$. Differentiate twice: $6au + 2b$, a straight line. So on every knot interval, f'' is a straight line; and since f is a cubic spline, f'' is continuous across the knots. A function that is straight on each interval and continuous at the joins is a continuous piecewise-linear function (a polyline), completely determined by its values at the knots.

Polylines have their own natural basis: the *hat functions*. The hat N_p is the polyline equal to 1 at knot p and 0 at every other knot: a triangle, rising over one interval and falling over the next. Every polyline is a combination of hats, with its knot-values as the coefficients. And hats are not a

new ingredient: they are exactly the *linear* B-splines on our knot vector (the three-knot windows of Section 2). That is why the previous section could deliver f'' directly in this basis: with $\boldsymbol{\gamma} = C\boldsymbol{c}$,

Why can any polyline be written in hats, and what are the coefficients?

Hats have one special property: at any knot, exactly one of them is nonzero, and its value there is 1: $N_p(t_q) = 1$ if $p = q$ and 0 otherwise. Take the combination whose coefficients are the polyline's own values at the knots, $g = \sum_p f''(t_p)\, N_p$, and evaluate it at a knot t_q: every hat vanishes except N_q, which contributes $f''(t_q) \cdot 1$. So g agrees with f'' at every knot; both are polylines, and a polyline is determined by its knot values, hence they are the same function:

$$f''(u) \;=\; \sum_{p=1}^{m+2} \gamma_p\, N_p(u), \qquad \gamma_p = f''(t_p), \qquad \boldsymbol{\gamma} = C\boldsymbol{c} \in \mathbb{R}^{m+2}. \tag{6}$$

The hat basis is *interpolatory*: its coefficients are function values. (Contrast the cubic basis, where the c_j are control heights, not values of f.) This also says what the entries of $\boldsymbol{\gamma} = C\boldsymbol{c}$ *mean*: the second derivative of the spline, evaluated at the knots.

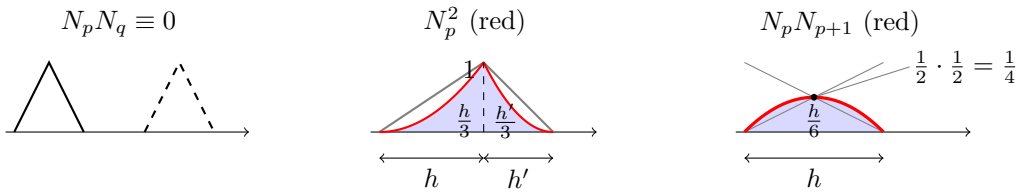
Squaring and integrating

Expand the square of the sum in the penalty (2):

$$\int (f'')^2 \,\mathrm{d}u \;=\; \sum_p \sum_q \gamma_p \gamma_q \underbrace{\int N_p(u)\, N_q(u)\,\mathrm{d}u}_{R_{pq}} \;=\; \boldsymbol{\gamma}^\top R\, \boldsymbol{\gamma}, \qquad R \in \mathbb{R}^{(m+2)\times(m+2)}. \tag{7}$$

The single integral reduces to a matrix of elementary integrals, $R_{pq} = \int N_p N_q$, called the *mass matrix* of the hat basis. Most entries vanish: two hats that are not neighbours have disjoint supports, their product is zero everywhere, and $R_{pq} = 0$. Only two kinds of entry survive, a hat with itself and a hat with its immediate neighbour, so R is tridiagonal.

The three cases, and two parabola areas



(a) $R_{pq} = 0$ (b) $R_{pp} = \frac{h}{3} + \frac{h'}{3}$ (c) area $= \frac{2}{3}\, h \cdot \frac{1}{4} = \frac{h}{6}$

Case (a) is the sparsity of R: hats that are not neighbours have disjoint supports, their product vanishes identically, and $R_{pq} = 0$. Only two kinds of entry survive, and both are areas under parabolas: straight sides multiplied by straight sides give quadratics.

Case (b), a hat with itself. On its rising interval of width h the hat is the ramp u/h, so N_p^2 is a parabola climbing from 0 to 1 with its vertex at the start. The area under such a parabola is one third of its bounding rectangle,

$$\tfrac{1}{3} \times h \times 1 = \tfrac{h}{3},$$

and the falling interval (width h') contributes $h'/3$ the same way:

$$R_{pp} \;=\; \frac{h+h'}{3} \;=\; \frac{t_{p+2}-t_p}{3}. \tag{8}$$

Case (c), a hat with its neighbour. They overlap on one interval, one falling as the other rises. Their product is zero at both ends and peaks at the midpoint, where both hats equal $\frac{1}{2}$: a parabolic arch of base h and height $\frac{1}{4}$. By Archimedes' rule the area under a parabolic arch is two thirds of its bounding rectangle,

$$\tfrac{2}{3} \times h \times \tfrac{1}{4} = \tfrac{h}{6},$$

so

$$R_{p,\,p+1} \;=\; \frac{t_{p+2}-t_{p+1}}{6}. \tag{9}$$

That is the entire computation of R: one third-rule and one two-thirds rule. Nothing is approximate, and no data has entered: both entries are read off the knot spacings.

The running example

Our ten knots carry 8 hats, so R is 8×8. But the first two and last two hats are built on windows of repeated boundary knots (widths $t_{p+2} - t_p = 0$): they are identically zero, and (8)–(9) produce zero rows for them automatically, with no special-casing. These zero rows match the zero rows of C in Section 2: they are coefficients of hats that do not exist. With interval widths 0.7, 1.6, 0.7:

$$R = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.2333 & 0.1167 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.1167 & 0.7667 & 0.2667 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.2667 & 0.7667 & 0.1167 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.1167 & 0.2333 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

As a check of one entry: $R_{44} = (t_6 - t_4)/3 = 2.3/3 = 0.7667$, the widest hat's width over three. And a global check: since the hats sum to one everywhere, row p of R must sum to $\int N_p = (t_{p+2} - t_p)/2$, half the hat's width. The row sums here are 0.35, 1.15, 1.15, 0.35 for the four live hats: exactly the half-widths.

The section's result, in one line: by (7), integrating the squared second derivative is a matrix sandwich, $\int (f'')^2 = \boldsymbol{\gamma}^\top R \boldsymbol{\gamma}$, with R known exactly from the knots alone. The next section only has to substitute $\boldsymbol{\gamma} = C\boldsymbol{c}$.

4 The penalty matrix

Sections 2 and 3 each produced one half of the computation: the matrix C turns the spline's coefficients into the coefficients of its second derivative, $\boldsymbol{\gamma} = C\boldsymbol{c}$, and the matrix R integrates the square of anything written in hat coefficients, $\int (f'')^2 = \boldsymbol{\gamma}^\top R \boldsymbol{\gamma}$. Substituting the first into the second eliminates $\boldsymbol{\gamma}$:

$$\int (f'')^2 \, \mathrm{d}u = (C\boldsymbol{c})^\top R \, (C\boldsymbol{c}) = \boldsymbol{c}^\top \big(C^\top R \, C\big) \boldsymbol{c}, \tag{10}$$

so, comparing with (3),

$$\boxed{\Omega = C^\top R \, C} \tag{11}$$

5 Using Ω: the penalized solve

The smoothing problem the implementation solves is

$$\min_{\boldsymbol{c}\in\mathbb{R}^m} \; (\boldsymbol{y} - X\boldsymbol{c})^\top W (\boldsymbol{y} - X\boldsymbol{c}) \; + \; \lambda\, \boldsymbol{c}^\top \Omega\, \boldsymbol{c}, \tag{12}$$

where $X \in \mathbb{R}^{n\times m}$, $X_{ij} = B_j(x_i)$, is the design matrix (n data points, m coefficients; rectangular, since $n \neq m$ in general), and $W = \mathrm{diag}(w) \in \mathbb{R}^{n\times n}$ holds the weights. Setting the gradient to zero gives the normal equations

$$(X^\top W X + \lambda\, \Omega)\, \boldsymbol{c} \;=\; X^\top W \boldsymbol{y}, \tag{13}$$

an $m \times m$ system. The matrix on the left has three properties that together select the solver:

- **Symmetric** (real Hermitian): $X^\top W X$ is symmetric by construction, and $\Omega = C^\top R C$ is symmetric because R is.
- **Positive definite** for $\lambda > 0$. Both terms are positive semi-definite because both quadratic forms are sums or integrals of squares: $\boldsymbol{c}^\top X^\top W X \boldsymbol{c} = \sum_i w_i \big(f(x_i)\big)^2 \geq 0$, and $\boldsymbol{c}^\top \Omega\, \boldsymbol{c} = \int (f'')^2 \geq 0$. Now suppose $\boldsymbol{c}$ is a null vector of the sum. Since neither term can be negative, both must vanish. The second vanishing, $\int (f'')^2 = 0$, forces $f'' \equiv 0$: the spline is a straight line. (This is also why $\Omega\mathbf{1} = 0$ and $\Omega\boldsymbol{g} = 0$ for the Greville vector $g_j = (t_{j+1} + t_{j+2} + t_{j+3})/3$, the coefficient vectors of the constant and linear functions.) The first vanishing then forces this line to be zero at every data point x_i; a straight line with two or more distinct roots is identically zero, so $\boldsymbol{c} = 0$. Hence, for any data set with at least two distinct x_i, the system matrix has no nonzero null vector: it is positive definite.
- **Banded** with bandwidth 3: cubic B-splines i and j have disjoint supports for $|i - j| > 3$, so both $X^\top W X$ and Ω are 7-banded.

A symmetric positive-definite banded system is solved by banded Cholesky factorization: in `scipy`, `solveh_banded` (the `h` is for Hermitian), which accepts only the $4 \times m$ unique lower bands and does roughly half the work of general banded LU. (The existing $t = x$ implementation uses the general solver `solve_banded` because it poses the problem in pre-divided form, $(X + \lambda\, wE)\boldsymbol{c} = \boldsymbol{y}$, the normal equations with $X^\top W$ cancelled from the left, where wE denotes its penalty term in the natural-spline basis, and that matrix is not symmetric. The same problem posed as normal equations *is* symmetric and could equally use the Cholesky solver, as the new path in fact does when $t = x$.)

6 Boundary conditions

Nothing in the construction so far imposes any behaviour at the ends of the interval: the minimization runs over all cubic splines on t, and the boundary does whatever the objective prefers. This section records how *natural* boundary conditions, zero curvature at both ends,

$$f''(t_{\min}) = 0, \qquad f''(t_{\max}) = 0, \tag{14}$$

fit into the same framework, should one wish to impose them.

The key observation is that (14) is already expressed in our matrices. By the interpolatory property (6), the entries of $\boldsymbol{\gamma} = C\boldsymbol{c}$ *are* the values of f'' at the knots, so (14) says simply that the first and last live entries of $C\boldsymbol{c}$ vanish: two linear constraints on $\boldsymbol{c}$, each involving only the two or three boundary rows of C that are nonzero there.

Linear constraints are imposed by *null-space reduction*. Let $Z \in \mathbb{R}^{m\times(m-2)}$ be a matrix whose columns span the coefficient vectors satisfying both constraints. Substituting $\boldsymbol{c} = Z\tilde{\boldsymbol{c}}$ turns the normal equations (13) into an $(m-2)\times(m-2)$ system in the reduced coefficients:

$$Z^\top\big(X^\top W X + \lambda\,\Omega\big)Z\,\tilde{\boldsymbol{c}} \;=\; Z^\top X^\top W\boldsymbol{y}, \tag{15}$$

after which $\boldsymbol{c} = Z\tilde{\boldsymbol{c}}$ recovers the B-spline coefficients. Because each constraint touches only a few boundary coefficients, Z differs from the identity only in its first and last few rows, and the reduced system remains banded.

Two remarks. First, this is not hypothetical infrastructure: the existing $t = x$ implementation in `scipy` *is* this construction. It works directly in a natural-spline basis of dimension $n = m-2$, and its basis-change map (eqs. (2.2.10) of Zemlyanoy (2022)) is exactly such a Z; we verified numerically, to 13 significant digits, that its penalty satisfies $X^\top W\,(wE) = Z^\top\Omega\,Z$ with Ω from (11): the existing penalty is ours, restricted to the natural subspace.

Second, the distinction between imposing and obtaining. At $t = x$, imposing (14) changes nothing: as shown in Section 9, the unconstrained minimizer satisfies the natural conditions on its own. For general knots $t \neq x$ this is no longer true: the constraint genuinely alters the solution, chiefly its behaviour near and beyond the boundaries (a natural spline extrapolates linearly; an unconstrained one leaves the interval as a full cubic).

7 Higher-degree splines: what would change

Suppose we wanted splines of degree $k > 3$ with the same penalty $\int(f'')^2$. Walking through the construction section by section, almost nothing changes, and the one thing that does has a clean, still-exact replacement.

What survives, and why. De Boor's formula (4) holds at every order, so the differentiation matrices, and hence C (now two reductions starting from degree k), are built exactly as in Section 2. The assembly $\Omega = C^\top RC$ never used "cubic" anywhere. The null space of Ω is still exactly the straight lines (lines have zero curvature at every degree), so the positive-definiteness argument of Section 5 and the invariant tests $\Omega\mathbf{1} = 0$, $\Omega\boldsymbol{g} = 0$ carry over verbatim. Bandedness survives too, with wider bands: basis functions overlap $|i-j| \le k$ neighbours, so Ω and $X^\top WX$ are $(2k+1)$-banded and `solveh_banded` applies unchanged. Even the oracle generalizes: `fda::bsplinepen` takes the order as an argument.

What breaks: the parabola shortcut. After two derivatives, f'' now lands in the degree-$(k-2)$ basis, not the hats. The products N_pN_q are no longer parabolas, so the third-rule and two-thirds-rule of Section 3, the entire computation of R, no longer apply, and no comparably short formula exists.

The replacement: exact quadrature. On each knot interval, N_pN_q is a single polynomial of known degree $2(k-2)$. A Gauss–Legendre rule with g points integrates any polynomial of degree $\le 2g-1$ with *zero* error, so choosing $g = k-1$ points per interval,

$$R \;=\; E^\top\,\mathrm{diag}(w)\,E \qquad E_{\ell p} = N_p(u_\ell), \tag{16}$$

with u_ℓ, w_ℓ the Gauss points and weights of all intervals, reproduces every entry $\int N_pN_q$ *exactly*: the same degree-matching logic as Simpson's rule in Wand and Ormerod (2008), one degree higher. The cost is one basis evaluation per point, $O(n)$ total, and the construction remains data-free. Indeed the cubic case is this same idea done once by hand: Archimedes' rule *is* the exact quadrature of a parabola, and (8)–(9) are its pre-computed answers.

In short: for $k > 3$, replace two hand-derived numbers by a $(k-1)$-point exact quadrature inside R, and touch nothing else. This is recorded here as a design note; the implementation is deliberately cubic-only.

8 What is from the literature, and what is derived here

The problem itself is classical. The penalized objective and the penalty matrix $\Omega_{ij} = \int B_i'' B_j''$ are standard in smoothing-spline theory (Wahba, 1990); using a B-spline basis with freely chosen knots and this exact integral penalty is known in the statistics literature as *O'Sullivan penalized splines* (Wand and Ormerod, 2008).

Ramsay and Silverman (2005, Ch. 5) discuss computing Ω and state that for B-spline bases the integrals "can be computed analytically," but decline to show how: "the details (see de Boor, 2002) are intricate, and few users of FDA will want to write programming code for this problem," referring readers instead to their software packages. The remainder of their section concerns numerical quadrature fallbacks. In other words, the standard reference names the gap this document fills, without filling it.

One prior exact construction is published. Wand and Ormerod (2008, eq. (3)) obtain Ω by observing that each $B_i'' B_j''$ is piecewise quadratic, so Simpson's rule with three evaluations per knot interval is *exact*: their $\Omega = (\widetilde{B}'')^\top \mathrm{diag}(w)\, \widetilde{B}''$ built from values of B'' at interval endpoints and midpoints. The present construction reaches the same matrix by a different route, coefficient algebra rather than evaluation: no quadrature nodes appear, and the banded structure and null space of Ω are visible directly in the factors C and R.

Equation by equation, the provenance is:

- **From the literature:** the derivative formula (4) (de Boor, 2001, p. 117); the hat-function integrals (8) and (9) (classical finite-element mass-matrix entries, rederived in Section 3 for completeness); the definition of the penalty (3) (Wahba, 1990).
- **Derived in this document:** the matrix form of differentiation (D_1, D_2) including the treatment of repeated knots; the composition (5) expressing f'' in the hat basis; and the assembly $\Omega = C^\top R C$, eq. (11). These steps were derived independently from (4); the `fda` package of Ramsay & Silverman implements an analytic Ω, but its source is GPL-licensed and was deliberately not consulted, only its numerical output was used, as a black-box test.

9 Validation

All checks are collected in a companion repository,

https://github.com/aadya940/scipy-bspline-testing

whose scripts are annotated with the equation numbers of this document and fail on any mismatch. The checks, and what each one certifies:

Check	Compared against	Result	Certifies
Ω on irregular knots	`fda::bsplinepen` (R)	10^{-6}	eq. (11)
Ω on irregular knots	direct quadrature of (3)	10^{-12}	eq. (11)
R closed form	quadrature of $\int N_p N_q$	10^{-10}	eqs. (8), (9)
Ω via Simpson exactness	Wand and Ormerod (2008, eq. (3))	10^{-8}	independent route
$\Omega\,\mathbf{1} = 0$	-	10^{-12}	null space = lines
symmetry, bandwidth 3	-	exact	structure of Ω
full solve, t = clamped x	existing `make_smoothing_spline`	10^{-14}–10^{-9}	entire pipeline

Two end-to-end checks exercise the full solve path in the proposed `scipy` implementation. At $\lambda = 0$ with sparse knots, the smoothing spline reproduces `make_lsq_spline` to 10^{-15}.

The second check places the knots at the clamped data sites, $t = x$, and compares against the *existing* `make_smoothing_spline`. To state precisely what is being tested, write the objective as

$$J(f) \;=\; \sum_{i=1}^{n} w_i \left(y_i - f(x_i)\right)^2 \;+\; \lambda \int (f'')^2 \,\mathrm{d}u, \tag{17}$$

and consider three nested sets of candidate functions:

$$\mathcal{S}_{\text{nat}} \;\subset\; \mathcal{S}_{\text{spl}} \;\subset\; \mathcal{F}, \tag{18}$$

where $\mathcal{F}$ is all twice-differentiable functions, $\mathcal{S}_{\text{spl}}$ is the cubic splines on the knots $t = x$ (dimension $n+2$), and $\mathcal{S}_{\text{nat}}$ is its natural-spline subspace, with $f'' = 0$ at both boundary knots (dimension n). The existing implementation minimizes J over $\mathcal{S}_{\text{nat}}$; the new path minimizes J over $\mathcal{S}_{\text{spl}}$, a strictly *more general* problem, with no boundary conditions imposed. In general a larger search set could find a strictly smaller minimum. But a classical result (Wahba, 1990) states that the minimizer over all of $\mathcal{F}$ is a natural cubic spline with knots at the data:

$$\hat{f} \;:=\; \arg\min_{f \in \mathcal{F}} J(f) \;\in\; \mathcal{S}_{\text{nat}}. \tag{19}$$

By (18), $\hat{f}$ lies in every one of the three sets, so minimizing over any of them returns the same function:

$$\arg\min_{\mathcal{S}_{\text{nat}}} J \;=\; \arg\min_{\mathcal{S}_{\text{spl}}} J \;=\; \hat{f}. \tag{20}$$

The natural boundary behaviour is not imposed on the new path; it *emerges* from the minimization, because end curvature costs penalty with no data beyond the boundary to justify it. Equation (20) is what the test verifies: the two implementations agree to machine precision at every tested λ (10^{-14} at $\lambda = 10^{-4}$, degrading only with conditioning to 10^{-9} at $\lambda = 100$), and any discrepancy beyond rounding would indicate a bug. This single test certifies the design matrix, the penalty (11), and the normal-equation solve simultaneously, with no external oracle.

The `fda` package was used strictly as a black-box oracle: only its numerical output was compared against; its GPL-licensed source was not consulted.

10 Conditioning at large λ

The question: the equivalence test of Section 9 agrees to 10^{-14} at small λ but only 10^{-9} at large λ. Why?
Symbols: κ the condition number (defined in Step 1), and $\varepsilon \approx 10^{-16}$ the machine precision.

Step 1: solving a linear system always loses digits. A computer carries about 16 digits ($\varepsilon \approx 10^{-16}$). When it solves $A\boldsymbol{c} = \boldsymbol{b}$, rounding errors get amplified by the condition number of A, and the answer is reliable only to

$$\text{error} \;\approx\; \varepsilon \cdot \kappa(A). \tag{21}$$

$\kappa(A)$ is the ratio of the largest to the smallest eigenvalue: how unevenly the matrix treats different directions. A matrix that is very stiff in some directions and very soft in others is the hard case.

Step 2: our matrix becomes exactly that as λ grows. The system we solve is

$$A_\lambda \;=\; X^\top W X + \lambda\,\Omega. \tag{22}$$

The second term grows with λ, but not in every direction, because Ω is blind to straight lines:

$$\Omega\,\boldsymbol{c}_{\text{line}} = \mathbf{0} \qquad \text{(lines have no curvature)}. \tag{23}$$

So in two directions of coefficient space (slope and intercept) the matrix keeps its fixed, λ-independent size, set by the data term alone, while in all $m-2$ others it grows like λ. Largest over smallest:

$$\kappa(A_\lambda) \;\propto\; \lambda. \tag{24}$$

Step 3: combine (21) and (24) and test. The prediction is error $\propto \lambda$, lockstep with κ, decade for decade. Measured (against a `float80` iterative-refinement reference; same data and knots throughout):

λ	$\kappa(A_\lambda)$	this implementation	`fda::smooth.basis`
10^{-4}	$5{\cdot}10^{3}$	$5{\cdot}10^{-16}$	$2{\cdot}10^{-15}$
10^{-2}	$5{\cdot}10^{4}$	$2{\cdot}10^{-15}$	$1{\cdot}10^{-13}$
1	$4{\cdot}10^{6}$	$4{\cdot}10^{-13}$	$6{\cdot}10^{-12}$
10^{2}	$4{\cdot}10^{8}$	$8{\cdot}10^{-11}$	$1{\cdot}10^{-9}$
10^{4}	$4{\cdot}10^{10}$	$2{\cdot}10^{-8}$	$3{\cdot}10^{-8}$

The answer. κ climbs seven decades and the error climbs seven decades with it: eq. (21) in action. The degradation is not two methods disagreeing; it is every solver losing the same digits to the same κ. The reference implementation of Ramsay and Silverman (2005) follows the same line, slightly above ours at every λ: the difficulty is intrinsic to the problem. It is also harmless: λ large enough to cost 10^{-8} produces a fit already indistinguishable from its straight-line limit.

11 Related software

A survey of what neighbouring ecosystems provide, and where the proposed implementation sits. The three Julia packages were inspected at source level; `fda` was compared numerically only (Section 10), its GPL source unread.

package	knots	penalty & construction	λ selection	solve
`scipy` (t=None)	data only	$\int (f'')^2$, divided diff.	GCV	banded LU
`SmoothingSplines.jl`	data only	$\int (f'')^2$, Reinsch	manual	banded Cholesky
`BSplineKit.jl`	data only	$\int (f'')^2$ exact, value-space	manual	banded Cholesky
`GeneralizedSS.jl`	data only	$\int (f^{(p)})^2$, RKHS kernel	marginal lik.	semisep. Cholesky
`fda` (R)	free	$\int (f'')^2$ exact	GCV	normal eqs.
`mgcv` (R)	free	O'Sullivan / P-spline	GCV / REML	reparam.
this work	**free**	$\int (f'')^2$ exact, $\Omega = C^\top R C$	explicit λ	banded Cholesky

Three takeaways from the survey.

1. No surveyed package outside R offers free knots. The three Julia packages fix the knots at the data. `SmoothingSplines.jl` is the classical Reinsch algorithm (Reinsch, 1967). `GeneralizedSS.jl` uses a kernel formulation with no B-splines at all; its "generalized" refers to higher-order penalties and shape constraints (monotonicity, convexity), not to knots. So the combination this work adds, free knots with an exact integral penalty outside R, exists nowhere in the surveyed ecosystems.

2. All surveyed packages use the same solver family. Each package factors a symmetric positive-definite system: banded Cholesky in `SmoothingSplines.jl` (LAPACK `pbtrf`, the same routine under `solveh_banded`), semiseparable Cholesky in `GeneralizedSS.jl`; `fda`'s documentation formulates the solve as normal equations (Ramsay and Silverman, 2005), and its numerical behaviour is consistent with that (Section 10), though, as that section shows, conditioning alone cannot distinguish the formulation. None of the source-inspected packages uses QR.

3. The same mass matrix everywhere. The tridiagonal matrix of eqs. (8)–(9) (diagonal $(h+h')/3$, off-diagonal $h/6$) appears verbatim as `ReinschR` in `SmoothingSplines.jl`, and (in its interior rows) as the grid-step matrix $\Delta/6$ in `BSplineKit.jl`'s smoothing fit, whose boundary rows are dropped because its natural boundary conditions zero the corresponding entries of $\boldsymbol{\gamma}$. Both packages apply it to second-derivative *values* at the knots, where this document applies it to hat-basis *coefficients*: the same numbers, by the interpolatory property of Section 3, since their knots sit at the data. Reinsch's 1967 algorithm and every implementation surveyed here share this single core object with eq. (11).

12 Selecting λ: generalized cross-validation

The default path selects λ automatically by generalized cross-validation (GCV); the user-knots path currently requires an explicit λ. This section derives the criterion from first principles, every step either derived in place or cited, and closes with what does and does not depend on the knot vector.

The fitted values are a matrix times the data

For fixed λ, the coefficients solve the normal equations (13). The matrix on the left is invertible for $\lambda > 0$ (Section 5, positive-definiteness argument), so we may write

$$\boldsymbol{c} = (X^\top W X + \lambda\Omega)^{-1} X^\top W \boldsymbol{y}. \tag{25}$$

Separately, evaluating the spline $f = \sum_j c_j B_j$ at data site x_i gives $\hat{y}_i = \sum_j c_j B_j(x_i)$, and the numbers $B_j(x_i)$ are precisely the entries of X (its definition). So all n evaluations at once are

$$\hat{\boldsymbol{y}} = X\boldsymbol{c}. \tag{26}$$

Substitute (25) into (26); this is literal substitution, nothing more:

$$\hat{\boldsymbol{y}} = X\Big[(X^\top W X + \lambda\Omega)^{-1} X^\top W\, \boldsymbol{y}\Big] = \underbrace{\Big[X\,(X^\top W X + \lambda\Omega)^{-1} X^\top W\Big]}_{=:\,A(\lambda)}\,\boldsymbol{y}. \tag{27}$$

$A(\lambda)$ is $n \times n$ and exists purely because (25) is linear in $\boldsymbol{y}$. Craven and Wahba (1979) introduce A by the same observation (p. 379): "Since $g_{n,\lambda}(t)$ is a linear function of $y_1, \ldots, y_n$ for each t, such $A(\lambda)$ exists."

What $\operatorname{tr} A$ means: degrees of freedom

We verify that $\operatorname{tr} A$ counts "parameters spent" in three cases where the answer is known independently.

Case A, interpolation. The fit passes through every point: $\hat{\boldsymbol{y}} = \boldsymbol{y}$ for every possible $\boldsymbol{y}$, which forces $A = I$, so

$$\operatorname{tr} A = \operatorname{tr} I = n \qquad \text{(all } n \text{ freedoms spent)}.$$

Case B, fitting the mean. Every $\hat{y}_i = \frac{1}{n}\sum_j y_j$, so each row of A is $(\frac{1}{n}, \ldots, \frac{1}{n})$, each diagonal entry is $\frac{1}{n}$, and

$$\operatorname{tr} A = n \cdot \tfrac{1}{n} = 1 \qquad \text{(one parameter spent)}.$$

Case C, unpenalized least squares on the m basis functions ($\lambda = 0$, with $m \le n$ and X of full column rank). The hat matrix $H = X(X^\top X)^{-1}X^\top$ is a projection ($H^2 = H$), and a projection's trace equals the dimension of the space it projects onto (Hastie et al., 2009, Sec. 3.2):

$$\operatorname{tr} H = m \qquad (m \text{ parameters spent}).$$

All three cases agree with the interpretation. Our smoothing A is none of these exactly: it *shrinks* each direction of the data by a factor in $[0, 1]$ rather than fully keeping or dropping it, so its trace is a non-integer between the extremes; hence *effective* degrees of freedom (Hastie and Tibshirani, 1990, Sec. 3.5); Craven and Wahba (1979) use $\operatorname{tr}(I - A)$ in this role in their eq. 1.9. Its limits follow from facts already proved: as $\lambda \to \infty$ the fit tends to the best straight line (lines span $\ker \Omega$, Section 5), a two-parameter fit, so $\operatorname{tr} A \to 2$; as $\lambda \to 0$ it tends to the least-squares spline on m basis functions (Case C), so $\operatorname{tr} A \to m$.

Computation. The trace is cyclic, $\operatorname{tr}(BC) = \operatorname{tr}(CB)$ (standard linear algebra). Apply it once to (27) with $B = X$ and $C = (X^\top W X + \lambda\Omega)^{-1}X^\top W$:

$$\operatorname{tr} A = \operatorname{tr}\left[(X^\top W X + \lambda\Omega)^{-1}\,(X^\top W X)\right]. \tag{28}$$

The left-hand side is $n \times n$; the right-hand side involves only the $m \times m$ banded matrices the solve already builds, so A is never formed. The same identity, for the same reason, appears in Wood (2017, Sec. 6.2), where the $m \times m$ product is denoted F and $\operatorname{tr}(A) = \operatorname{tr}(F)$ is noted as the computationally preferable form.

Leave-one-out cross-validation and its shortcut

To score how well a given λ generalizes: hide one point, fit without it, predict it, average the misses over all points:

$$V_0(\lambda) = \frac{1}{n}\sum_{k=1}^{n}\left(g^{[k]}(x_k) - y_k\right)^2, \tag{29}$$

where $g^{[k]}$ is the spline fitted to the $n - 1$ points with (x_k, y_k) removed (Craven and Wahba, 1979, eq. 1.11). As written this costs n refits. Two lemmas remove the cost.

Lemma 1 (Craven and Wahba, 1979, Lemma 3.1, proved there by a three-line inequality argument): $g^{[k]}$ is also the solution of the *full* n-point problem in which y_k is replaced by the value $g^{[k]}(x_k)$. Intuition: $g^{[k]}$ already passes through that value at x_k; adding back a data point lying exactly on the curve changes nothing.

Lemma 2 (Craven and Wahba, 1979, Lemma 3.2): by (27), $\hat{y}_k$ depends linearly on y_k with slope $a_{kk} = \partial\hat{y}_k/\partial y_k$, the k-th diagonal entry of A. Applying Lemma 1 and this linearity (fit with the modified y_k = old fit + slope × modification):

$$g^{[k]}(x_k) = \hat{y}_k + \big(g^{[k]}(x_k) - y_k\big)\, a_{kk}.$$

Subtract y_k from both sides and solve for the leave-one-out miss $g^{[k]}(x_k) - y_k$:

$$g^{[k]}(x_k) - y_k = \frac{\hat{y}_k - y_k}{1 - a_{kk}}. \tag{30}$$

Substituting (30) into (29) gives (Craven and Wahba, 1979, eq. 3.1):

$$V_0(\lambda) = \frac{1}{n}\sum_{k=1}^{n} \frac{(\hat{y}_k - y_k)^2}{(1 - a_{kk})^2}. \tag{31}$$

Exact leave-one-out from *one* fit: each residual inflated by its *leverage* a_{kk}, i.e. by how much y_k pulls its own fitted value toward itself (the full fit is already drawn toward high-leverage points, so their residuals are magnified to compensate).

From leave-one-out to GCV

Replace every individual leverage in (31) by the average leverage,

$$a_{kk} \;\longrightarrow\; \frac{1}{n}\sum_k a_{kk} = \frac{1}{n}\operatorname{tr} A.$$

The denominator becomes constant across the sum and the numerator collapses to the residual sum of squares (Craven and Wahba, 1979, eq. 1.9):

$$V(\lambda) = \frac{\frac{1}{n}\|(I - A)\boldsymbol{y}\|^2}{\big(1 - \frac{1}{n}\operatorname{tr} A\big)^2} = \frac{\mathrm{RSS}(\lambda)/n}{\big(1 - \mathrm{df}(\lambda)/n\big)^2}. \tag{32}$$

Why is the replacement justified? Two answers, both in Craven and Wahba (1979).

Formal (Craven and Wahba, 1979, Sec. 3, p. 387): we exhibit a rotated coordinate system in which the GCV score (32) *is* the ordinary leave-one-out score (31). Take $W = I$ for this subsection (the weighted case is discussed under *Assumptions* below); then

$$A = X(X^\top X + \lambda\Omega)^{-1}X^\top$$

is symmetric, since transposing it reproduces it term by term ($(X^\top X + \lambda\Omega)$ is symmetric, and the inverse of a symmetric matrix is symmetric). By the spectral theorem, a symmetric matrix has an orthonormal basis of eigenvectors: writing them as the *columns* of U (so $U^\top U = I$ and $Au_k = d_k^2 u_k$),

$$A = UD^2U^\top, \qquad D^2 = \operatorname{diag}(d_1^2, \ldots, d_n^2), \tag{33}$$

where the notation d_k^2 records that the eigenvalues are nonnegative (CW compute them as squared singular values, p. 387). In our setting, A has rank at most m (it factors through the m-dimensional coefficient space), so at most m of its eigenvalues are nonzero: these take the explicit form $d_k^2 = 1/(1 + \lambda\rho_k) \in (0, 1]$ for $k = 1, \ldots, m$, where $\rho_k \geq 0$ is the *roughness* of the k-th eigendirection, the value of the penalty per unit squared amplitude in that direction, with $\rho_k = 0$ for the two

straight-line directions (which the penalty never touches) and ρ_k large for strongly oscillating ones (Section 13, where these are derived by simultaneous diagonalization); the remaining $n-m$ eigenvalues are zero. Equivalently $U^\top A\,U = D^2$: entry (j,k) of $U^\top AU$ is $u_j^\top A u_k = d_k^2\, u_j^\top u_k = d_k^2\delta_{jk}$. This is a change of basis only; A acts in the new coordinates by scaling axis k by d_k^2.

Now the second rotation. Let Φ be the $n\times n$ *Fourier matrix*: its entry in row j, column k $(j,k=1,\dots,n)$ is

$$\Phi_{jk} = \frac{1}{\sqrt{n}}\, e^{2\pi ijk/n},$$

a complex number of absolute value $1/\sqrt{n}$. For a complex matrix M, the symbol M^* denotes its *conjugate transpose*: transpose M and replace every entry by its complex conjugate, so $(M^*)_{jk} = \overline{M_{kj}}$; for a real matrix it is just the transpose. In particular $(\Phi^*)_{kl} = \overline{\Phi_{lk}} = \frac{1}{\sqrt{n}}\, e^{-2\pi ilk/n}$. The Fourier matrix satisfies $\Phi\Phi^* = I$ (it is *unitary*, the complex analogue of orthogonal; the proof is the standard geometric-series computation $\sum_k e^{2\pi i(j-l)k/n} = 0$ for $j\neq l$ and $=n$ for $j=l$). Define the composite rotation $\Gamma = \Phi\, U^\top$ (unitary as a product of unitaries; this is CW's Γ, p. 387). Rotate the data and the fit: $\tilde{\boldsymbol{y}} = \Gamma\boldsymbol{y}$, $\tilde{\hat{\boldsymbol{y}}} = \Gamma\hat{\boldsymbol{y}}$. Then, using $\hat{\boldsymbol{y}} = A\boldsymbol{y}$ (27) and (33),

$$\tilde{\hat{\boldsymbol{y}}} = \Gamma A\boldsymbol{y} = \Gamma A\Gamma^*\tilde{\boldsymbol{y}} = \Phi\, U^\top (UD^2U^\top)\, U\Phi^*\,\tilde{\boldsymbol{y}} = \underbrace{\Phi D^2\Phi^*}_{=:\,\tilde A}\;\tilde{\boldsymbol{y}}, \tag{34}$$

so in the rotated coordinates the prediction matrix (the matrix carrying rotated data to rotated fitted values, the analogue of A) is $\tilde A = \Phi D^2\Phi^*$. Compute its entry in row j, column l directly from the rule for multiplying three matrices (sum over the inner index k; D^2 is diagonal, so only its diagonal entries d_k^2 appear):

$$\tilde A_{jl} = \sum_{k=1}^{n} \Phi_{jk}\, d_k^2\, (\Phi^*)_{kl} = \sum_{k=1}^{n} \frac{e^{2\pi ijk/n}}{\sqrt{n}}\, d_k^2\, \frac{e^{-2\pi ilk/n}}{\sqrt{n}} = \frac{1}{n}\sum_{k=1}^{n} d_k^2\, e^{2\pi i(j-l)k/n}, \tag{35}$$

where the last step multiplies the two exponentials: $e^{2\pi ijk/n}\cdot e^{-2\pi ilk/n} = e^{2\pi i(j-l)k/n}$ (add the exponents), and $\frac{1}{\sqrt{n}}\cdot\frac{1}{\sqrt{n}} = \frac{1}{n}$. The result depends on the row and column numbers (j,l) only through their difference $j-l$: every diagonal of $\tilde A$ is constant ($\tilde A$ is *circulant*). In particular, setting $j=l$ makes the exponential $e^0 = 1$, so every leverage in the rotated system equals

$$\tilde a_{jj} = \frac{1}{n}\sum_{k=1}^{n} d_k^2\, e^0 = \frac{1}{n}\sum_{k=1}^{n} d_k^2 = \frac{1}{n}\operatorname{tr} A, \tag{36}$$

the same number for every j. The last equality uses $\operatorname{tr} A = \sum_k d_k^2$: the trace is unchanged by a change of basis ($\operatorname{tr}(UD^2U^\top) = \operatorname{tr}(D^2U^\top U) = \operatorname{tr} D^2$ by the cyclic property already used in (28)), and the trace of the diagonal matrix D^2 is the sum of its diagonal entries. Now perform ordinary leave-one-out (31) *in the rotated system*: with all leverages equal to (36), the denominator is the constant $(1-\frac{1}{n}\operatorname{tr} A)^2$ and factors out of the sum,

$$\tilde V_0(\lambda) = \frac{1}{n}\sum_{k} \frac{|\tilde{\hat y}_k - \tilde y_k|^2}{(1-\tilde a_{kk})^2} = \frac{\frac{1}{n}\|(I-\tilde A)\tilde{\boldsymbol{y}}\|^2}{(1-\frac{1}{n}\operatorname{tr} A)^2} = \frac{\frac{1}{n}\|(I-A)\boldsymbol{y}\|^2}{(1-\frac{1}{n}\operatorname{tr} A)^2} = V(\lambda), \tag{37}$$

where the middle step is $\|(I-\tilde A)\tilde{\boldsymbol{y}}\| = \|\Gamma(I-A)\boldsymbol{y}\| = \|(I-A)\boldsymbol{y}\|$, because $(I-\tilde A)\tilde{\boldsymbol{y}} = \Gamma(I-A)\Gamma^*\Gamma\boldsymbol{y} = \Gamma(I-A)\boldsymbol{y}$ and unitary matrices preserve norms. So GCV is not an approximation of cross-validation; it *is* cross-validation, carried out in a rotated coordinate system whose leverages are all equal. The

"generalized" records this rotation-invariance: raw leave-one-out (31) changes under rotations of the data (the a_{kk} do), while V does not (RSS and $\operatorname{tr} A$ are both rotation-invariant).

Assumptions used (an explicit inventory). The derivation (33)–(37) used exactly one structural fact: *symmetry of* A. It used no property of the knot vector t and no spacing property of the data x; the knots live inside the entries of X and Ω and never surface. Equal spacing does appear in CW, but in two *other* places: (i) their motivating special case (Craven and Wahba, 1979, Lemma 2.2 and Sec. 2), where for periodic, equally spaced data A is circulant already in the *original* coordinates, so ordinary CV equals GCV with no rotation needed (a special case, not a hypothesis of the argument above); and (ii) the asymptotic optimality theory (Craven and Wahba, 1979, Thm. 4.3 via Lemma 4.2, p. 391), whose hypotheses are about how the *data points* x_i are spread out: roughly, that as n grows they fill the interval without leaving gaps or piling up (formally, CW require them to be placed according to a fixed positive density), plus a condition on eigenvalue growth (their A4.3.1) that CW verify only informally, for data that is close to evenly spaced. Two caveats for our setting. First, with nontrivial weights $A = X(X^\top W X + \lambda\Omega)^{-1}X^\top W$ is *not* symmetric (only $W^{1/2}A\,W^{-1/2}$ is), so the argument above applies verbatim to $W = I$ and otherwise must be run in the W-inner product (equivalently, applied to the symmetric matrix $W^{1/2}AW^{-1/2}$ acting on $W^{1/2}\boldsymbol{y}$). Second, CW's optimality theorem is proved for their setting (knots at the data, basis size growing with n); our fixed-m, user-chosen-t regime is a different asymptotic regime, that of penalized regression splines (Wood, 2017, Sec. 2), so we adopt V as the selection criterion with the same leave-one-out meaning, not on the strength of Thm. 4.3.

Asymptotic (Craven and Wahba, 1979, Thm. 4.3, p. 391): the λ minimizing V achieves mean-square error within a factor tending to 1 of the best possible λ as $n \to \infty$ (in CW's knots-at-data regime; see the second caveat above); their measured inefficiencies on test problems were 1.01–1.42 (Craven and Wahba, 1979, Sec. 5, Table 1).

Interpretation of (32). If the fit spends everything ($\mathrm{df} \to n$), the denominator $\to 0$ and V diverges, no matter how small RSS becomes: a fit that reproduces the noise is penalized by construction. This is precisely the role of the denominator.

Assembly, and what depends on $t \neq x$

The pipeline, each arrow justified above:

$$\text{solve (13)} \xrightarrow{\text{substitute}} \hat{\boldsymbol{y}} = A\boldsymbol{y} \xrightarrow{\text{Lemmas 1–2}} \text{LOOCV (31)} \xrightarrow{\text{rotation}} V(\lambda) \xrightarrow{\text{minimize}} \hat{\lambda}.$$

The two computable ingredients are below. The $W^{1/2}$ is only notation for the weighted residual sum: since $W = \operatorname{diag}(w)$, its square root is $W^{1/2} = \operatorname{diag}(\sqrt{w})$, and

$$\sum_i w_i\,(y_i - \hat{y}_i)^2 = \sum_i \left(\sqrt{w_i}\,(y_i - \hat{y}_i)\right)^2 = \|W^{1/2}(\boldsymbol{y} - X\boldsymbol{c})\|^2,$$

i.e. exactly the fidelity term of the objective (12) (with $w_i = 1$ it reduces to the unweighted $\|(I - A)\boldsymbol{y}\|^2$ of Craven and Wahba (1979)). So

$$\mathrm{RSS} = \|W^{1/2}(\boldsymbol{y} - X\boldsymbol{c})\|^2, \qquad \mathrm{df} = \operatorname{tr}\left[(X^\top W X + \lambda\Omega)^{-1}X^\top W X\right] \text{ by (28)},$$

each evaluation of V costing one banded solve and one banded trace. For the minimization, Craven and Wahba (1979) evaluated V on a grid of $\log_{10} p$ (their $p = 1/n\lambda$) "in increments of 1/9" and took the global minimum (their Sec. 5): a log-grid search, in the founding paper.

What depends on the knots. Inspect (25)–(32): knots appear only *inside* the entries of X and Ω. No equation used the shape of X; (25) needed only invertibility, which holds for any valid clamped

t (Section 5). The leave-one-out construction (29)–(31) mentions data points and entries of A only. So the criterion is identical for any knots. The existing implementation differs in one shortcut,

$$\boldsymbol{y} - X\boldsymbol{c} = \lambda\, wE\, \boldsymbol{c},$$

obtained by rearranging the *square* system of the knots-at-data formulation (Zemlyanoy, 2022, eq. 2.2.4); for rectangular X no such rearrangement exists and RSS must be computed as written. Additionally the trace (28) runs at size m instead of n. That is the entire difference.

13 The spectrum of A: attenuation of rough components

Section 12 constructed $A(\lambda)$ as a single matrix. This section describes its action on the data: A resolves $\boldsymbol{y}$ into components, retains the smooth ones, and attenuates the rough ones, with λ setting the scale at which attenuation begins. The statements below make this description exact. Throughout this section $W = I$; for general weights every statement holds with the Euclidean inner product replaced by the W-inner product, as in the *Assumptions* inventory of Section 12.

A common eigenbasis. The two matrices appearing in the normal equations (13), the data term $X^\top X$ and the penalty term Ω, are both symmetric, and (for $X^\top X$ positive definite) a classical result states that they can be diagonalized *simultaneously* (Demmler and Reinsch, 1975); see also Hastie and Tibshirani (1990, Sec. 3.5). That is, there exists a single basis $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m$ of coefficient space in which both quadratic forms are diagonal. In this basis, direction i carries a single number $\rho_i \geq 0$, the value of the penalty per unit squared amplitude in that direction, which we call its *roughness*. Ordering the basis so that $\rho_1 \leq \cdots \leq \rho_m$, the directions are ranked from smoothest to roughest. The first two correspond to the constant and the linear function, and carry no penalty:

$$\rho_1 = \rho_2 = 0 \qquad \text{(linear functions have zero curvature; } \ker\Omega\text{, Section 5).}$$

In the periodic, equally spaced case these directions are the Fourier modes, with ρ_ν growing polynomially in the frequency ν (Craven and Wahba, 1979, Sec. 3).

Decoupling. In this basis the minimization decouples into m independent scalar problems. In direction i, let a denote the amplitude favoured by the data term and z the amplitude of the fit; the objective restricted to this direction is the one-variable quadratic

$$(z - a)^2 + \lambda\rho_i\, z^2.$$

Setting its derivative to zero,

$$2(z - a) + 2\lambda\rho_i z = 0 \qquad \Longrightarrow \qquad z = \frac{a}{1 + \lambda\rho_i},$$

so the fit retains the fraction

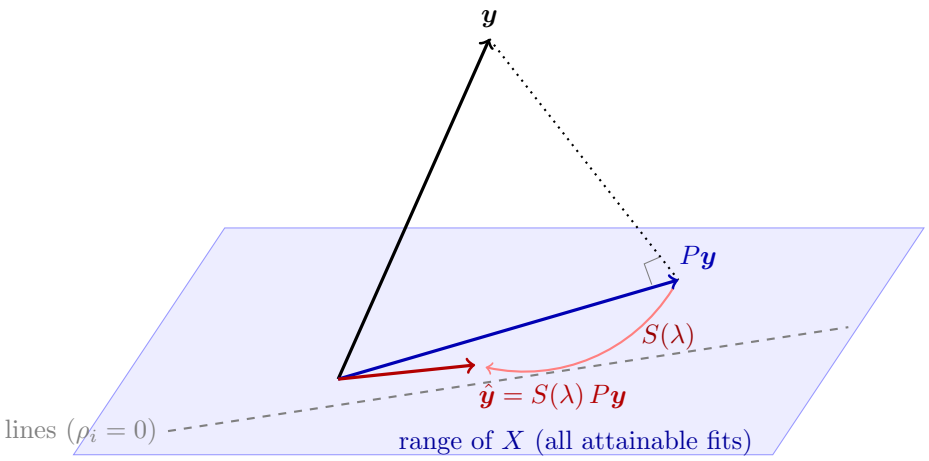
$$d_i(\lambda) = \frac{1}{1 + \lambda\rho_i} \tag{38}$$

of the amplitude favoured by the data in direction i. Interpretation: an unpenalized direction ($\rho_i = 0$) is retained in full, $d_i = 1$, for every λ; a direction with $\lambda\rho_i$ large is attenuated almost entirely. The d_i are the nonzero eigenvalues of $A(\lambda)$; Craven and Wahba (1979, Sec. 3) compute exactly these quantities as filter coefficients in their Fourier case.

Factorization of A. Assembling the directions, A acts as a composition of two operations:

$$A(\lambda) = S(\lambda)\, P, \tag{39}$$

where P is the orthogonal projection of $\boldsymbol{y}$ onto the range of X (the subspace of attainable fitted vectors), and $S(\lambda)$ scales the component along direction i within that subspace by $d_i(\lambda)$. The limiting cases confirm the factorization: at $\lambda = 0$ every $d_i = 1$, so $S = I$ and $A = P$, the least-squares projection; as $\lambda \to \infty$ every direction with $\rho_i > 0$ is annihilated and A tends to the projection onto the linear functions.

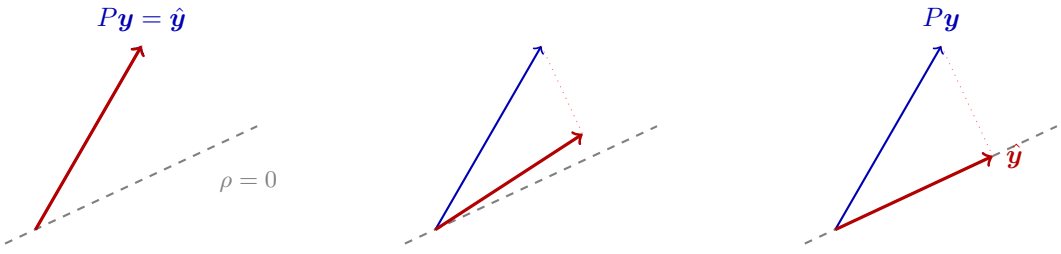


The factorization (39) *as geometry: P drops $\boldsymbol{y}$ perpendicularly onto the subspace of attainable fits (the range of X); $S(\lambda)$ then moves the result within that subspace, attenuating each direction by $d_i(\lambda)$, toward the unpenalized line directions as λ grows.*

The action of $S(\lambda)$ alone is easiest to see from directly above the plane. Decompose $P\boldsymbol{y}$ along the in-plane eigenbasis: a component along the unpenalized line direction ($\rho = 0$) and a component along a rough direction with roughness $\rho > 0$. By (38) the first is retained in full and the second is multiplied by $1/(1 + \lambda\rho)$, so

$$\hat{\boldsymbol{y}}(\lambda) \;=\; \underbrace{(P\boldsymbol{y} \cdot \boldsymbol{u}_1)\,\boldsymbol{u}_1}_{\text{kept}} \;+\; \frac{1}{1+\lambda\rho}\,\underbrace{(P\boldsymbol{y} \cdot \boldsymbol{u}_2)\,\boldsymbol{u}_2}_{\text{attenuated}}. \tag{40}$$

Three snapshots of (40), viewed from above the plane (the line direction dashed; $P\boldsymbol{y}$ blue, fixed; $\hat{\boldsymbol{y}}$ red, moving):



$\lambda\rho = 0$ $\qquad$ $\lambda\rho = 4$ $\qquad$ $\lambda\rho \to \infty$

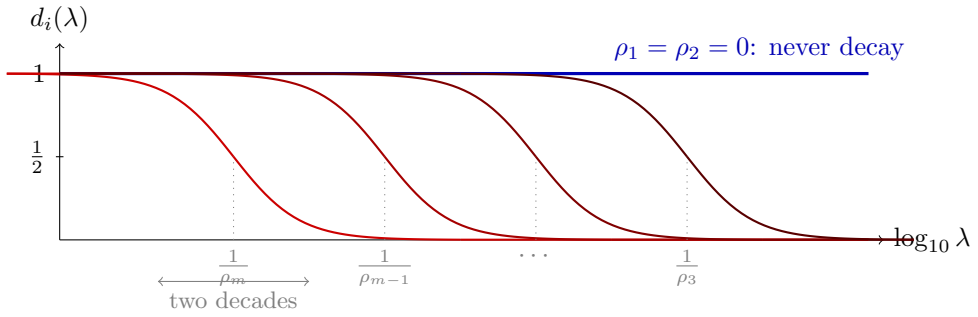
At $\lambda = 0$ the fit is the projection itself; as $\lambda\rho$ grows, the rough component of $\hat{\boldsymbol{y}}$ shrinks by $1/(1+\lambda\rho)$ while the line component is untouched, and in the limit $\hat{\boldsymbol{y}}$ lies on the line direction: the df $\to 2$ *endpoint of* (41).

Degrees of freedom. Taking the trace of (39),

$$\mathrm{df}(\lambda) = \operatorname{tr} A = \sum_{i=1}^{m} d_i(\lambda) = \sum_{i=1}^{m} \frac{1}{1+\lambda\rho_i}. \tag{41}$$

Since d_i is the retained fraction of direction i, df is the total retained amplitude across directions: a fully retained direction contributes 1, a fully attenuated one contributes 0, and a direction at intermediate attenuation contributes its fraction. At $\lambda = 0$ the sum equals m. As λ increases, the contribution of direction i decays once λ exceeds the direction's characteristic scale $1/\rho_i$ (substituting $\lambda = 1/\rho_i$ into (38) gives $d_i = \frac{1}{2}$). Rough directions have small characteristic scales and decay first; the two unpenalized directions never decay. The sum therefore decreases continuously from m to 2, which establishes the limits of df stated in Section 12.

Width of the GCV minimum. The rate of decay of a single direction follows from evaluating (38) on either side of its characteristic scale: at $\lambda = 0.1/\rho_i$ the retained fraction is $1/(1+0.1) \approx 0.91$, while at $\lambda = 10/\rho_i$ it is $1/(1+10) \approx 0.09$. The transition from largely retained to largely attenuated therefore spans a factor of 100 in λ, i.e. two decades, for every direction. Moreover, the characteristic scales $1/\rho_i$ of different directions are themselves spread over many decades. The criterion $V(\lambda)$ is assembled from RSS and df, both of which change only through these transitions; a quantity composed of many two-decade transitions occurring at staggered positions cannot vary abruptly. Consequently $V(\lambda)$ varies slowly along the $\log\lambda$ axis and its minimum lies in a trough several decades wide rather than in a narrow spike. The design of the λ search rests on this property.



Each attenuation factor d_i decays from 1 to 0 over two decades of λ, centred at its own characteristic scale $1/\rho_i$; the scales are staggered across many decades, and the two unpenalized directions never decay. $\mathrm{df}(\lambda)$ *is the sum of these curves, and $V(\lambda)$ inherits their slow, staggered variation.*

Conditioning. The matrix inverted by the solve, $X^\top W X + \lambda\Omega$, has eigenvalues proportional to $1 + \lambda\rho_i$ in the same basis, the reciprocals of the attenuation factors (38). Its condition number is the ratio of the extreme eigenvalues:

$$\kappa = \frac{1+\lambda\rho_m}{1+\lambda\rho_1} = 1 + \lambda\rho_m \;\propto\; \lambda, \tag{42}$$

since $\rho_1 = 0$. The unpenalized directions and the most strongly attenuated direction separate linearly in λ: strong smoothing and good conditioning are structurally in tension. This is the same $\kappa \propto \lambda$ law measured in Section 10, rederived from the attenuation factors. Note that the ill-conditioning resides in the solve, which inverts this spread; applying A itself is benign, its eigenvalues being the bounded $d_i \in [0, 1]$.

References

P. Craven and G. Wahba. Smoothing noisy data with spline functions: estimating the correct degree of smoothing by the method of generalized cross-validation. *Numerische Mathematik*, 31:377–403, 1979.

C. de Boor. B(asic)-spline basics. Technical report, 1986. https://ftp.cs.wisc.edu/Approx/bsplbasic.pdf

C. de Boor. *A Practical Guide to Splines*, revised edition. Springer, New York, 2001.

A. Demmler and C. Reinsch. Oscillation matrices with spline smoothing. *Numerische Mathematik*, 24:375–382, 1975.

P. J. Green and B. W. Silverman. *Nonparametric Regression and Generalized Linear Models: A Roughness Penalty Approach*. Chapman & Hall/CRC, 1993.

T. Hastie and R. Tibshirani. *Generalized Additive Models*. Chapman & Hall, London, 1990.

T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd edition. Springer, New York, 2009.

N. M. Patrikalakis, T. Maekawa, and W. Cho. *Shape Interrogation for Computer Aided Design and Manufacturing*. Hyperbook edition. https://web.mit.edu/hyperbook/Patrikalakis-Maekawa-Cho/

J. O. Ramsay and B. W. Silverman. *Functional Data Analysis*, 2nd edition. Springer, New York, 2005. Ch. 5, "The roughness penalty approach."

C. H. Reinsch. Smoothing by spline functions. *Numerische Mathematik*, 10(3):177–183, 1967.

G. Wahba. *Spline Models for Observational Data*. SIAM, Philadelphia, 1990.

M. P. Wand and J. T. Ormerod. On semiparametric regression with O'Sullivan penalized splines. *Australian & New Zealand Journal of Statistics*, 50(2):179–198, 2008.

H. J. Woltring. A Fortran package for generalized, cross-validatory spline smoothing and differentiation. *Advances in Engineering Software*, 8(2):104–113, 1986.

S. N. Wood. *Generalized Additive Models: An Introduction with R*, 2nd edition. Chapman & Hall/CRC, 2017.

E. Zemlyanoy. Generalized cross-validation smoothing splines. BSc thesis, 2022. In Russian. https://www.hse.ru/ba/am/students/diplomas/620910604